NafaPilot

Engineering handoff for the next developer or LLM

Project state, file structure, rules and next steps (after Milestone 9)

Team PayMatrix · Paytm Build for India AI Hackathon 2026 · Track 1

Read this first. This document lets an AI assistant or a new developer continue the project without the original conversation. It covers the rules that must not be broken (chapter 2), how the user works (chapter 3), every file and what it does (chapter 4), the invariants and where they're enforced (chapter 5), schemas, APIs and algorithms (chapters 6–8), the frontend (chapter 9), commands (chapter 10), known pitfalls (chapter 11), the backlog and the next milestone (chapter 12), and a ready-to-paste starter prompt (chapter 13).

State as of 29 October 2026. Milestones M1–M8 were independently audited (M8: "pass with caveats"). M9 (the merchant dashboard) is delivered and waiting for review. All data is synthetic, and execution is simulated. This is not an official Paytm product.

Contents

1. Project summary and status
2. Non-negotiable rules
3. How the user works
4. File structure (every file)
5. Invariants and where they are enforced
6. Database schema
7. API reference
8. Engines: formulas, algorithms, constants
9. Frontend architecture
10. Commands
11. Pitfalls and lessons learned
12. Known gaps, backlog and M10
13. Starter prompt for the next LLM

1. Project summary and status

NafaPilot is a profit-aware merchant growth decision engine. Pipeline: **detect** a sales change → the **LLM proposes** campaign candidates → **Python calculates** incremental profit scenarios → the **policy engine** can veto → the **merchant approves** exact inputs → **simulated execution** with an 80/20 holdout → **measure** uplift with a confidence interval → explicit **feedback** (preferences).

M	Milestone	Status	Main code
1	Project setup	Done	`backend/app/main.py` , `config.py` , `frontend/`
2	LLM provider (Gemini + mock)	Done	`services/llm/`
3	Synthetic dataset	Done	`app/synthetic/` , `scripts/generate_dataset.py`
4	Analytics and baseline (+ review fixes)	Done	`services/analytics/`
5	Profit and policy engines	Done	`services/decision/`
6	Database and campaign API	Done	`db/` , `campaign_store.py` , migration 0001
7	Recommendations, feedback, explanations	Done	`recommendations.py` , migration 0002
8	Simulated execution and measurement	Done (audit: pass with caveats)	`execution.py` , `services/experiments/` , migration 0003
9	Merchant dashboard	Delivered, awaiting review	`frontend/`
10	End-to-end tests, deployment, offline fallback	Not started	—

Stack: Python 3.11+ (developed on 3.13), FastAPI, Pydantic v2, pydantic-settings, SQLAlchemy 2, Alembic, psycopg 3, PostgreSQL 16 (SQLite in tests), pandas, NumPy, SciPy, google-genai. Frontend: Next.js 16.4.3, React 19.3, TypeScript 5, Tailwind v4, Vitest 5 + Testing Library. Quality: ruff, mypy --strict, ESLint, tsc.

Paths: the user keeps the project at `D:\GithHub\hackathons\PAYMATRIX` (Windows, PowerShell). Planned hosting: Render (backend).

2. Non-negotiable rules

These come from the user's master prompt and each milestone request. Breaking them is a failed milestone.

Product and integrity

- **Synthetic data only; simulated execution only.** Never call Paytm or any real messaging, offer or payment service. Never present synthetic or simulated results as real observations.

- **Never claim to be an official Paytm product.** Keep the disclaimer in the UI, README and API (`/api/v1/meta`).
- **The LLM never decides.** It proposes candidates and words explanations. Profit, policy, selection and verdicts are deterministic code. The LLM can't override or bypass policy.
- **Never infer causes.** Opportunity alerts always report `cause: "unknown"` .
- **Report `insufficient_data` instead of guessing.** Never default a missing input, never treat missing as zero, never invent customer IDs.
- **Keep number kinds separate:** observed, projection, prediction, simulated measurement. Never overwrite predictions or approval/evaluation snapshots.
- **Explicit preferences only.** Free-text rejection reasons are never turned into preferences.

Engineering

- Preserve the architecture and existing API contracts. No microservices, no agent frameworks, no model training or fine-tuning. Ask before major architectural changes or destructive operations.
- **Do not change M5 accounting formulas** or silently redefine campaign-budget scope (budget applies to C only).
- **Decimal for all money** in the backend. Round only final values (ROUND_HALF_UP to paise).
- **No secrets** in source, frontend bundles, logs, audit payloads, fixtures or chat. Keys live only in the git-ignored root `.env` or the host's environment settings.
- Reversible Alembic migrations that preserve existing data; constraints and transactions for integrity.
- Default tests must not call live Gemini (the `live` marker is excluded by default).

Process (from the user's milestone prompts)

- **Do not run Git commands or inspect Git.** (A `.git` folder exists in the workspace; leave it alone.)
- Read the relevant existing code and docs before implementing; follow existing conventions; don't duplicate existing infrastructure.
- Run only relevant tests, not the full suite, unless justified. **Never claim a test passed unless it was actually run.** Report anything the environment prevented.
- **Stop after the requested milestone** and wait for review. Don't start the next one.

3. How the user works

- Work is split into milestones. Each milestone request is detailed and lists scope, constraints, required tests and the final report format. An independent auditor ("Antigravity") reviews each milestone and returns PASS / PASS WITH CAVEATS / corrections. Review requests ask for targeted fixes only.
- **Final report format** the user expects: changed files, migrations, APIs, method and assumptions, exact test commands and results, limitations, then "stopped, waiting for review".
- **Delivery:** zip the project with `zip` (not git) and send it. Exclude `.git` , `.venv` , `node_modules` , `.next` , caches, `data/synthetic/` , `.env` , `.env.local` , `*.db` , `tsconfig.tsbuildinfo` and older zips. The user extracts it over the Windows folder, so the dataset must be regenerated after extracting.
- The user prefers clear, plain explanations and asks for documents (PDFs) explaining the work.
- The Claude Code VS Code extension didn't work for the user (403 errors), so work happens in a chat session with a cloud workspace.

4. File structure (every file)

Root

```
PAYMATRIX/
├── .env.example            all backend settings (copy to .env); frontend var noted at the bottom
├── .gitignore              secrets, venvs, node_modules, local DBs, generated data
├── README.md               setup, checks, API table, LLM setup, milestones
├── docker-compose.yml      local PostgreSQL 16 (user/pass/db = nafapilot)
├── data/README.md          dataset tables, simulation model, planted scenarios
├── data/synthetic/         GENERATED CSVs + manifest.json (not committed, not zipped)
├── docs/
│   ├── decision-engine.md  M5 formulas, policy rules, interpretations to review
│   ├── persistence.md      M6 tables, workflow rules, refusals
│   ├── recommendations.md  M7 run flow, grounding, preferences
│   ├── experiments.md      M8 eligibility, idempotency, assignment, estimator
│   ├── frontend.md         M9 pages, idempotency in the UI, integration notes
│   └── NafaPilot_Build_Report_M1-M9.pdf
└── scripts/
    ├── generate_dataset.py  writes data/synthetic (default seed 42; --seed, --out)
    └── check_gemini.py      lists models / live ping + sample for the configured key
```

Backend: `backend/`

```
alembic.ini, pyproject.toml (pytest/ruff/mypy config; addopts excludes "live")
requirements.txt, requirements-dev.txt
app/
├── main.py              FastAPI app, CORS, exception handlers (LLMError, DatasetMissing,
│                        UnknownMerchant, CampaignError → {error_code, message, details?}), /health
├── config.py            Settings (pydantic-settings; env files .env and ../.env); SecretStr key
├── logging_config.py    log redaction of secret values
├── api/v1/
│   ├── router.py        includes all routers; GET /meta
│   ├── llm.py           GET /llm/status, POST /llm/ping (404 in production)
│   ├── merchants.py     GET /merchants, /merchants/{id}, /merchants/{id}/baseline
│   ├── data.py          POST /data/validate
│   ├── campaigns.py     POST /campaigns/evaluate (stateless) + persisted campaign CRUD/decision/audit
│   ├── recommendations.py  POST /merchants/{id}/recommendations, GET /recommendations/{id},
│   │                    POST /recommendations/{id}/explanations, GET /merchants/{id}/preferences
│   └── experiments.py   POST .../versions/{n}/execute (Idempotency-Key), experiment reads
├── schemas/             Pydantic contracts: analytics, decision, campaigns, llm,
│                        recommendations, experiments
├── data/repository.py   DataRepository: loads CSVs (from_dir) or tables (from_tables, tests);
│                        merchants, policies, orders, customers, historical_holdout()
├── db/
│   ├── base.py          declarative Base + naming convention
│   ├── models.py        all ORM models; _IMMUTABLE / APPEND_ONLY_TABLES; before_flush guard
│   └── session.py       make_engine (SQLite FK pragma), make_sessionmaker, get_session
├── services/
│   ├── analytics/       validation.py, baseline.py, opportunities.py, analyzer.py (pure)
│   ├── decision/        profit.py, policy.py, adapters.py (LLM candidate → CampaignInput) (pure)
│   ├── experiments/     assignment.py, simulation.py, measurement.py (pure)
│   ├── llm/             base.py (Protocol), factory.py, gemini.py, mock.py, prompts.py
│   │                    (PROMPT_VERSION "m7-v1"), grounding.py, templates.py, errors.py
│   ├── campaign_store.py   M6 workflow: create/version/decide/read/audit; CampaignError family
│   ├── recommendations.py  M7 run flow, selection rule, preferences, explanations
│   └── execution.py        M8 execute (locks, idempotency, refusals, audit) + experiment reads
└── synthetic/           generator.py, specs.py (planted scenarios)
migrations/
├── env.py               SQLite: PRAGMA foreign_keys OFF during migrations (batch rebuilds)
└── versions/0001_campaign_persistence.py, 0002_recommendations_preferences.py,
```

```
            0003_simulated_execution.py
tests/
├── conftest.py, test_health.py, test_dataset.py
├── analytics/ (validation, analysis, API)   decision/ (profit, policy, evaluate API, fixtures)
├── llm/ (gemini provider with fake transport, mock, grounding, schemas, redaction,
│         endpoints, test_live_gemini.py = opt-in live test)
├── experiments/ (assignment+simulation, measurement: pure unit tests)
└── db/ (conftest: SQLite always, PostgreSQL if TEST_DATABASE_URL; campaigns,
         recommendations, experiments, migrations)
```

Frontend: `frontend/`

```
AGENTS.md            Next.js 16 note: read node_modules/next/dist/docs before coding
.env.example         NEXT_PUBLIC_API_BASE_URL=http://localhost:8000 (copy to .env.local)
next.config.ts       cacheComponents + partialPrefetching enabled; Tailwind turbopack loader
package.json         scripts: dev, build, start, lint, typecheck (next typegen && tsc), test (vitest
run)
vitest.config.mts    jsdom, tests/setup.ts, env NEXT_PUBLIC_API_BASE_URL=http://api.test
app/
├── layout.tsx       header, skip link, main, footer (disclaimer)
├── globals.css      Tailwind + .btn-*, .field, .link, .kind-hatch, focus-visible
├── page.tsx         merchant picker + backend/LLM status
├── merchants/[merchantId]/page.tsx       → MerchantDashboard
├── recommendations/[runId]/page.tsx      → RunReview
├── campaigns/[campaignId]/page.tsx       → CampaignReview
└── experiments/[experimentId]/page.tsx   → ExperimentResults
   (each page: server component awaiting params inside , renders a client component)
lib/
├── api.ts           ALL types (mirror backend schemas) + ApiError + every endpoint function;
│                    runSimulation(campaignId, version, idempotencyKey, overrides);
newIdempotencyKey()
├── useApi.ts        data hook: abort on change, reload(), keeps data during refresh
├── format.ts        ₹/number/date formatting, labels (null → "—", never 0)
├── messages.ts      error_code → plain-language title/action; policy rule labels
└── status.ts        campaign stage derived from backend fields
components/
├── ui.tsx           KindBlock/KindTag (number-kind markers), Panel, ErrorBox, Badge, Loading, Empty
├── charts.tsx       DailyOrdersChart, IntervalChart (SVG; always shows zero)
├── AppHeader.tsx, BackendStatus.tsx, LlmStatus.tsx
├── merchant/        MerchantPicker, MerchantDashboard, BaselinePanel, Opportunities,
│                    RecommendPanel, CampaignsPanel
├── recommendation/  RunReview (selection, candidates, explanations, ask again)
├── campaign/        CampaignReview, WorkflowRail, Prediction (PredictionBlock, PolicyBlock),
│                    DecisionForm, EditVersionForm, SimulatePanel (only caller of execute)
└── experiment/      ExperimentResults (ExperimentView), AssignmentsTable
tests/
├── setup.ts         jest-dom, mocks next/link and next/navigation
├── helpers.ts       fixture loader, mockFetch, deferred
├── fixtures/*.json  REAL responses captured from the running M8 backend
└── api.test.ts, campaign.test.tsx, experiment.test.tsx
```

5. Invariants and where they are enforced

Invariant	Enforced by
Approval binds to exact inputs	Composite FK `(campaign_version_id, input_hash)` → `campaign_versions(id, input_hash)`; decision requires the submitted hash
One decision per version; decisions only on the current version	UNIQUE on approvals; `campaign_store.record_decision` (stale_version, already_decided)
Approval can't override policy	`record_decision` refuses approval when not permitted; execution re-checks policy with the current merchant policy
History is immutable	No update/delete code; ORM `before_flush` guard (`ImmutableRecordError`); DB triggers on every append-only table
A version executes at most once	UNIQUE `executions.campaign_version_id`; row lock; service checks
Idempotent execution	UNIQUE `executions.idempotency_key`; same key + version → replay; other version → `idempotency_key_reused`
Executed campaigns can't get new versions	`create_version` raises `already_executed`
Holdout never treated	CHECK on outcomes (holdout ⇒ redemptions = 0); composite FK outcome → assignment group
Execution mode is simulated only	CHECK `mode = 'simulated'`; campaign status CHECK in (`not_executed`, `simulated`)
Explanations grounded or template	DB CHECK `source = 'template' OR grounded`; `grounding.py`
Evaluations are predictions	CHECK `result_kind = 'prediction'`; CHECK `NOT executable OR permitted`
Engines don't touch the LLM or I/O	Package structure; a test checks decision modules' imports
Failed writes leave nothing	`with session.begin()` per operation; refusals and failures audited in a separate transaction
Downgrade can't silently lose executions	Migration 0003 downgrade raises if any campaign is `simulated`

6. Database schema

Migration	Tables	Notes
0001_campaign_persistence	`campaigns` (mutable pointer), `campaign_versions`, `campaign_evaluations`, `campaign_approvals`, `audit_events`	Append-only except campaigns; PG function `nafapilot_forbid_mutation` + triggers; SQLite `RAISE(ABORT)` triggers
0002_recommendations_preferences	`recommendation_runs`, `explanations`, `merchant_preferences`; `campaigns.recommendation_run_id` (nullable FK)	Batch alter on SQLite with FK enforcement off
0003_simulated_execution	`executions`, `assignments`, `outcomes`, `measurements`; campaigns status CHECK widened	Assignments PK (execution_id, customer_id); outcomes nullable orders/redemptions (missing outcomes); measurement result JSON

`merchant_id` is an indexed column, not a foreign key: merchants live in the CSV dataset (`DataRepository`), and the API validates IDs there. Model–migration parity is tested with Alembic `compare_metadata` (diff must be empty).

7. API reference

Base: `/api/v1`. Errors: `{error_code, message, details?}`; FastAPI validation errors use `{detail: [...]}`.

Method	Path	Notes
GET	`/health` (no prefix), `/meta`	Liveness; data-source labels
GET/POST	`/llm/status`, `/llm/ping`	Never returns the key; ping 404 in production
GET	`/merchants`, `/merchants/{id}`, `/merchants/{id}/baseline`	503 `dataset_missing` if CSVs absent; 404 `merchant_not_found`
POST	`/data/validate`	Validation report; stores nothing
POST	`/campaigns/evaluate`	Stateless M5 evaluation
POST/GET	`/campaigns`, `/campaigns?merchant_id=`, `/campaigns/{id}`	CampaignOut has `approval_status`, `permitted`, `executable`, `execution_status`, `experiment_id`, `current` (VersionOut), `notice`
GET/POST	`/campaigns/{id}/versions`, `/versions/{n}`	POST = edit (full CampaignInput) → new version; 409 `no_change`, `already_executed`
POST	`/campaigns/{id}/versions/{n}/decision`	`{decision, input_hash, decided_by, reason, preference?}`; 409 `stale_version`, `input_hash_mismatch`, `already_decided`, `policy_blocked`
GET	`/campaigns/{id}/audit`	Append-only events; `campaign_created` payload has `recommendation_run_id`
POST	`/merchants/{id}/recommendations`	`{campaign_costs_inr: {type: cost}, lang, requested_by}` → RecommendationRunOut (201)
GET/POST	`/recommendations/{run_id}`, `/recommendations/{run_id}/explanations`	Explain again: `{lang, question}`
GET	`/merchants/{id}/preferences`	Latest explicit preference per key
POST	`/campaigns/{id}/versions/{n}/execute`	Header `Idempotency-Key` (8–128 chars). Body `{requested_by, simulation: {true_lift_per_treated_customer, base_rate_per_customer, base_redeem_share, missing_outcome_share, max_customers}}`. 201 new, 200 replay. 409 `not_approved`, `stale_version`, `policy_blocked`, `already_executed`, `idempotency_key_reused`; 422 `insufficient_data`, `policy_missing`
GET	`/campaigns/{id}/experiments`, `/experiments/{id}`, `/experiments/{id}/assignments?group=&limit=&offset=`	ExperimentOut sections: approval, assignment, simulation, prediction, observed, incremental, measured_profit, verdicts, caveats, notice

JSON money: analytics/decision/campaign responses use numbers; stored `inputs` use Decimal strings; experiment `measured_profit` values are 2-dp strings. The frontend types in `lib/api.ts` reflect this.

8. Engines: formulas, algorithms, constants

M4 analytics

- Rates are per day *with recorded orders*; gaps = 3+ consecutive days without orders (cause unknown).
- Baseline = last 28 calendar days to `as_of`. Reference = first 4 of the last 12 weeks; recent = last 2 weeks.
- Exact binomial test per segment; Benjamini–Hochberg at 2.5% FDR; effect gate ≥10% in the same direction for the latest week and last 2 weeks; confidence "high" if the 8-week trend agrees ($p < 0.05$) and no week has missing days.
- Projection = latest weekly rate × the segment's calendar days in 4 weeks (assumes trading every day), labelled `is_projection`.

M5 profit and policy

```
ΔR = I×(AOV−d) − S×d ;  ΔP = ΔR − I×VC − C ;  I* = ceil((S×d + C)/(AOV−d−VC))   (exact fractions)
Rules: valid_inputs, discount_cap (d/AOV×100 ≤ cap), min_post_discount_margin ((AOV−d−VC)/(AOV−d)×100
≥ min),
       max_loss (ΔP[risk_scenario] ≥ −max_loss), campaign_budget (C ≤ budget), merchant_approval
(execution only)
Limits inclusive. permitted = all recommendation rules pass; executable = permitted AND approved for
this hash.
Deck regression: A = +100 / +2,400 / +4,700 (break-even 26); B = −3,600 / +1,200 / +5,400 (break-even
100, blocked by max_loss).
```

M7 selection rule

Among permitted candidates: max expected ΔP → max conservative ΔP → min discount. If none has expected ΔP > 0 → "no action". Campaign cost comes only from the request (`campaign_costs_inr`).

M8 experiments

```
seed        = sha256("nafapilot-experiment:{version_id}:{input_hash}")
cap         = smallest sha256("{seed}:population:{cid}") when max_customers is set
split       = order by sha256("{seed}:split:{cid}"); first round_half_up(n×holdout_pct/100) = holdout
eligibility = targeted_offer/off_peak: identified customers with orders_in_window ≥ 1; new_customer: =
0
simulation  = rng(int(seed[:16],16)); base~Poisson(b); extra~Poisson(max(L,0)); L<0 →
Poisson(max(b+L,0));
              redemptions = extra + Binomial(base, redeem_share); missing → NULL
defaults    = b from merchant's historical holdout mean (none → 422 unless supplied);
              L = expected I / treated; redeem_share = clip(expected S/(treated×b), 0, 1)
estimator   = Welch difference in means (scipy ttest_ind equal_var=False), 95%; Î = diff × treated
profit      = Î×(AOV−VC) − R×d − C (R scaled to all treated under missing-at-random)
verdicts    = decisive only if interval excludes 0 AND min measured per arm ≥ 30
(MIN_MEASURED_PER_ARM)
execution   = refused if fewer than 30 assigned per arm
```

9. Frontend architecture

- **Next.js 16 specifics:** `params` is a Promise. With `cacheComponents` on, each dynamic page awaits params inside a `<Suspense>` and renders a client component. Read `node_modules/next/dist/docs/` before using unfamiliar APIs (AGENTS.md).
- **ESLint React rules are strict:** no ref reads during render, no synchronous setState inside effects. `useApi` uses `useEffectEvent` and derives the loading state from a request key.

- **All HTTP goes through `lib/api.ts`.** Add new endpoints there with types mirroring the backend schema. `ApiError` has `status` (0 = unreachable), `code`, `details`, `retryable` (0 or ≥ 500).
- **The frontend never calculates authoritative values.** Verdicts and intervals are shown exactly as returned; nulls render as "—", "missing" or "no data".
- **Simulation:** only `SimulatePanel` calls execute. A new key for each confirmed run; the same key on "Retry the same request" after a network or 5xx error; the key is cleared on a 4xx, on cancel, or when assumptions change. A ref guard plus a disabled button prevent double submits.
- **Styling:** Tailwind utility classes plus a few component classes in `globals.css`. Number-kind markers in `components/ui.tsx`. System fonts, no web fonts (works offline).
- **Tests** stub `fetch` and use the real captured fixtures; `next/link` and `next/navigation` are mocked in `tests/setup.ts`. Don't select `[role=alert]` in browser automation: Next's route announcer also uses it; use `data-testid="api-error"`.

10. Commands

```
# one-time (project root)
Copy-Item .env.example .env
docker compose up -d                                  # or a local/hosted PostgreSQL

# backend
cd backend; python -m venv .venv; .\.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt
python ..\scripts\generate_dataset.py                 # REQUIRED after every fresh extract
python -m alembic upgrade head
python -m uvicorn app.main:app --reload --port 8000

# frontend
cd frontend; Copy-Item .env.example .env.local; npm install; npm run dev

# checks
python -m pytest [path]; python -m ruff check .; python -m mypy app
$env:TEST_DATABASE_URL="postgresql+psycopg://nafapilot:nafapilot@localhost:5432/nafapilot_test"   #
disposable DB only
npm run lint; npm run typecheck; npm test; npm run build

# optional live Gemini test (needs key in .env)
$env:RUN_LIVE_LLM=1; python -m pytest -m live
```

Last verified results (29 Oct 2026): backend 242 passed, 80 skipped (PostgreSQL variants), 1 deselected (live); PostgreSQL run of tests/db + tests/experiments: 175 passed, 2 skipped; frontend 25 passed; lint, types and build clean.

11. Pitfalls and lessons learned

Pitfall	What to do
503 on `/merchants` after extracting the zip	The dataset isn't shipped. Run `generate_dataset.py`.
PostgreSQL tests leave state; 0003 downgrade refuses with executions	The test fixture resets PG by `DROP SCHEMA public CASCADE; CREATE SCHEMA public`, not by downgrading. Never point it at real data.
SQLite batch migrations break foreign keys	`migrations/env.py` turns `PRAGMA foreign_keys` off during migrations and back on.
Cloud sandbox: the PostgreSQL server stops between sessions	Restart it with `pg_ctl` (data dir `/tmp/claude-0/pgdata`, socket `/tmp/claude-0/pgsock`, run as user `claude`); this applies only to the assistant's workspace.
`pkill -f` killed its own shell	Kill by port/PID, not by a pattern that matches the current command.
Vitest install failed on peer deps	`@types/node` was raised to ^22 for Vitest 5.
Google Fonts break offline builds	System fonts only.
Naive p < 0.01 gave false alerts (M4)	Benjamini–Hochberg plus an effect-size gate.
Deck interval "≈0 to 96" isn't reproduced by Welch	The deck was illustrative; the method wasn't tuned to match it (reported to the user).
Zero-effect simulations sometimes come out "positive"	Expected ~5% false positives (checked: about 4% over 400 null runs). Not a bug; explain it.
Low power for M001	MDE ≈ 107 extra orders (80% power) vs. expected 60: most default runs are inconclusive. Candidate improvement: power check before execution.

12. Known gaps, backlog and M10

Known gaps

- No endpoint lists a merchant's recommendation runs (the dashboard links a campaign to its run through the audit trail).
- `CampaignSummary` has no approval status (the dashboard fetches details for the 20 most recent campaigns).
- Interpretations awaiting confirmation: post-discount margin definition; campaign budget applies to C only.
- The profit interval treats redemptions as fixed; Welch is approximate for skewed counts.
- No authentication or roles (prototype).

Improvement backlog (not built; see the pitch guide)

Power/minimum-detectable-effect check before execution · demo reset script · predicted-vs-measured summary card · run-list endpoint · calibrate I/S from past experiments · CUPED variance reduction · probability of profit (bootstrap) · pooled estimates across similar merchants · message preview (not sent).

M10 (from the README roadmap): end-to-end tests, deployment and offline fallback

Not specified in detail yet; wait for the user's M10 request. Likely contents: Playwright end-to-end test of the demo flow; Render backend (build installs requirements and generates the dataset; pre-deploy `alembic upgrade head`; start `uvicorn app.main:app --host 0.0.0.0 --port $PORT`); hosted PostgreSQL (Neon/Aiven/Render) via `DATABASE_URL` with `postgresql+psycopg://…?sslmode=require`; Vercel frontend with `NEXT_PUBLIC_API_BASE_URL`; `CORS_ORIGINS` updated; offline fallback (mock LLM, local DB).

13. Starter prompt for the next LLM

Paste this, together with this PDF and the project zip, into a new session:

```
You are continuing NafaPilot, a profit-aware merchant growth decision engine for the Paytm Build for
India AI Hackathon 2026 (Team PayMatrix, Track 1). Milestones M1–M9 are complete; M9 (merchant
dashboard) awaits review. Read the attached handoff PDF first, then README.md and docs/*.md, then the
code relevant to the task.

Rules (do not break):
- Synthetic data and simulated execution only. Never call Paytm or real services. Never present
  synthetic/simulated results as real. Never claim to be an official Paytm product.
- The LLM only proposes candidates and words explanations. Profit, policy, selection and verdicts are
  deterministic code. Never infer causes; report insufficient_data instead of guessing.
- Do not change M5 accounting formulas or budget scope. Decimal for money. No secrets anywhere
  except the git-ignored .env / host env vars.
- Preserve architecture and API contracts. No microservices, agent frameworks or model training.
  Reversible migrations that preserve data.
- Do not run Git commands. Run only relevant tests; never claim a test passed unless you ran it.
- Default tests must not call live Gemini.
- Work only on the milestone requested. At the end report: changed files, migrations, APIs,
  methods/assumptions, exact test commands and results, limitations. Then stop and wait for review.
- Deliver the project as a zip (made with zip, excluding .git, .venv, node_modules, .next, caches,
  data/synthetic, .env, .env.local, *.db) and remind the user to regenerate the dataset.

Setup in the workspace: backend venv, pip install -r requirements-dev.txt,
python ../scripts/generate_dataset.py, alembic upgrade head; frontend npm install.
First task: wait for the user's review of M9 or their M10 request.
```